

Taller APIs - Plan de ejecución

Objetivo y alcance

Validar end-to-end el espacio de datos `demo` desde la API:

- **Provider** publica vocabulario, políticas, asset y contract definition.
- **Consumer** descubre el catálogo, inspecciona ofertas y negocia el contrato.
- **Transfer** ejecuta la transferencia, sigue su ciclo de vida, resuelve el EDR y accede de verdad al recurso.

Esta página es la hoja de ruta del taller, los requisitos previos, las variables compartidas y el smoke test. Los `curl` concretos viven dentro de cada fase enlazada más abajo.

Requisitos previos

Antes de empezar el taller debe estar operativo el entorno local de `ds-local-deployment`. La instalación completa, paso a paso para macOS, está aquí:

[Guía de despliegue en Local \(macOS\)](#).

Checklist mínimo de arranque

- ☒ ~~Docker Desktop arrancado.~~
- ☒ ~~Gluster Minikube en marcha.~~
- ☒ ~~Addon de Ingress habilitado.~~
- ☒ ~~Túnel de minikube activo en una terminal aparte.~~
- ☒ ~~Reenvíos de puerto activos para PostgreSQL y Vault.~~
- ☒ ~~Vault con unseal hecho.~~
- ☒ ~~Pods sanos en los namespaces common-srvs y demo.~~
- ☒ ~~Hosts locales con las entradas de los servicios y los conectores.~~
- ☒ ~~Herramienta jq instalada.~~

Verificación rápida del entorno

Antes de empezar la Fase 1 conviene comprobar que el cluster responde y que los pods de los namespaces `common-srvs` y `demo` están en estado `Running`.

```
minikube status
minikube kubectl -- get pods -n common-srvs
minikube kubectl -- get pods -n demo
```

Variables de entorno del taller

Este bloque debe cargarse al inicio de cada terminal nueva. Todos los `curl` posteriores reutilizan estas variables. El `SUFFIX` evita choques de IDs si el taller se repite varias veces.

```
# === Dataspace ===
export DS_NAME="test3"
export DS_DOMAIN="ds.inesdata-project.eu"
export KEYCLOAK_URL="http://admin.auth.ds.inesdata-project.eu"
export KC_CLIENT="dataspace-users"
```

```
# === Provider ===
export PROVIDER="conn-citycouncil-demo"
export PROVIDER_HOST="${PROVIDER}.${DS_DOMAIN}"
export PROVIDER_BASE="http://${PROVIDER_HOST}"
export PROVIDER_USER="user-conn-citycouncil-demo"
export PROVIDER_PASSWORD='iFgSxxz_K!YrA4t3'
export PROVIDER_PROTOCOL="http://${PROVIDER}:19194/protocol"
```

```
# === Consumer ===
export CONSUMER="conn-company-demo"
export CONSUMER_HOST="${CONSUMER}.${DS_DOMAIN}"
export CONSUMER_BASE="http://${CONSUMER_HOST}"
export CONSUMER_USER="user-conn-company-demo"
export CONSUMER_PASSWORD='e1miuSVEmhh7Vx@H'
```

```
export CONSUMER_PROTOCOL="http://${CONSUMER}:19194/protocol"
}
```

```
# === Sufijo único para esta corrida (evita choques de IDs)
===
export SUFFIX="$(date +%s)"
echo "Suffix de esta sesión: $SUFFIX"
```



AVISO: El `counterPartyAddress` siempre es la URL **interna** del cluster (`http://${PROVIDER}:19194/protocol`), nunca la pública con `dev.ds.dataspaceunit.upm` . El conector consumidor hace la llamada DSP desde dentro del cluster.

Se crean archivos ejecutables por cada uno de esos bloques. Los nombres de los archivos son `export_<nombre_bloque>.sh` . Posteriormente deben ajustarse los permisos.

```
chmod +x <nombre_archivo>.sh
```

Al inicio del taller:

```
source /Users/jpardo/Desktop/Proyectos/INESdata/data_spaces/demos/API/export_dataspace.sh
source /Users/jpardo/Desktop/Proyectos/INESdata/data_spaces/demos/API/export_suffix.sh
```

Estos dos son la **base común**:

- `export_dataspace.sh` contiene las variables globales del espacio (DS_NAME, KEYCLOAK_URL, etc.)
- `export_suffix.sh` genera el timestamp único para toda la sesión (CRÍTICO: ejecutarlo una sola vez para que todos los IDs compartan el mismo sufijo)

Por fase:

Fase 1: Provider

```
source /Users/jpardo/Desktop/Proyectos/INESdata/data_spaces/demos/API/export_provider.sh
```

Solo se necesitan las variables del provider porque esta fase trabaja con `conn-citycouncil-demo`.

Fase 2: Consumer

```
source /Users/jpardo/Desktop/Proyectos/INESdata/data_spaces/demos/API/export_consumer.sh
```

Aquí se añaden las del consumer (`conn-company-demo`).



AVISO: las variables del provider siguen disponibles porque son necesarias para `PROVIDER_PROTOCOL` y `PROVIDER_PARTICIPANT_ID` en los comandos de catálogo.

Fase 3: Transfer

No es necesario ejecutar nada nuevo; provider y consumer ya quedan cargados desde las fases anteriores.



AVISO: El `SUFFIX` solo se genera una vez al principio, porque todos los IDs del taller (`asset-${SUFFIX}` , `vocab-${SUFFIX}` , etc.) deben compartir el mismo timestamp para que el flujo sea coherente.

Endpoints

```
source /Users/jpardo/Desktop/Proyectos/INESdata/data_spaces/demos/API/endpoints.sh
```

Smoke test inicial

Pequeña verificación de que el management responde antes de empezar la Fase 1. Si esto falla, es necesario volver a la guía de despliegue.



COMANDOS: Renovar JWT como primer reflejo ante errores

Los JWT de Keycloak caducan en pocos minutos. Si **cualquier** `curl` devuelve HTML en lugar de JSON (típicamente página 401 "Unauthorized" o 502 "Bad Gateway"), o `jq` falla con `parse error: Invalid numeric literal`, lo primero es **regenerar el JWT** del rol en uso antes de buscar otra causa:

```
export PROVIDER_JWT=$(curl -s -X POST "${KEYCLOAK_URL}/realms/${DS_NAME}/protocol/openid-connect/token" \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "grant_type=password" \
-d "client_id=${KC_CLIENT}" \
-d "username=${PROVIDER_USER}" \
-d "password=${PROVIDER_PASSWORD}" \
-d "scope=openid profile email" | jq -r .access_token)

export CONSUMER_JWT=$(curl -s -X POST "${KEYCLOAK_URL}/realms/${DS_NAME}/protocol/openid-connect/token" \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "grant_type=password" \
-d "client_id=${KC_CLIENT}" \
-d "username=${CONSUMER_USER}" \
-d "password=${CONSUMER_PASSWORD}" \
-d "scope=openid profile email" | jq -r .access_token)
```

Igualmente, después de un `rollout restart` de los conectores es necesario esperar 30-60 s a que el EDC complete el arranque interno y regenerar los JWT (los anteriores apuntan a una sesión de Keycloak que el conector ya no reconoce).

```
export PROVIDER_JWT=$(curl -s -X POST \
  "${KEYCLOAK_URL}/realms/${DS_NAME}/protocol/openid-connect/token" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "grant_type=password" \
  -d "client_id=${KC_CLIENT}" \
  -d "username=${PROVIDER_USER}" \
  -d "password=${PROVIDER_PASSWORD}" \
  -d "scope=openid profile email" | jq -r .access_token)

echo "PROVIDER_JWT: ${PROVIDER_JWT:0:40}..."
```

Resultado esperado:

- Debe verse algo como: `PROVIDER_JWT: eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUI...` (los primeros 40 caracteres del token)
- Si aparece `PROVIDER_JWT: ...` (vacío o "null"), significa que Keycloak no respondió o las credenciales son incorrectas

```
curl -s -X POST "${PROVIDER_BASE}/management/v3/assets/request" \
  -H "Authorization: Bearer ${PROVIDER_JWT}" \
  -H "Content-Type: application/json" \
  -d '{"@context":{"@vocab":"https://w3id.org/edc/v0.0.1/ns/"},"offset":0,"limit":1,"filterExpression":[]}' \
  | jq .
```

Resultado esperado:

Si el conector está limpio (sin assets previos), debería devolver un **array vacío**:

```
[ ]
```

Si ya hay assets creados, aparecerá algo como:

```
[
  {
    "@id": "asset-123456789",
```

```
[{"@type": "Asset",
  "properties": {
    "name": "...",
    ...
  },
  "dataAddress": {
    ...
  }
}]
```

Interpretación del smoke test

✓ Smoke test OK si:

- El `PROVIDER_JWT` no está vacío (tiene un token JWT largo)
- La llamada a `/management/v3/assets/request` devuelve **200 OK** con un JSON (sea array vacío `[]` o con datos)

✗ Smoke test FAILED si:

- `PROVIDER_JWT` está vacío o es "null" → problema de autenticación Keycloak
- La llamada a assets devuelve error HTTP (401, 403, 404, 500, timeout) → problema con el conector o la red

El objetivo del smoke test es confirmar **antes de empezar** que:

1. Keycloak responde y autentica correctamente
2. El conector provider está levantado y su Management API es accesible



ERROR: Si el despliegue se ha realizado en varias fases, o el EDD se ha reiniciado en varias ocasiones, puede que los componentes se hayan degradado.

El conector está en **estado degradado**: aunque Kubernetes lo marca como `Running`, **el proceso Java EDC dentro del contenedor no está escuchando en el puerto 19193** (Management API). Los logs anteriores muestran un patrón claro:

1. El conector intenta arrancar
2. Falla al obtener las keys de Keycloak (cuando éste está en 503)
3. Crashea con `Failed to get keys from <http://keycloak.dev.ed.dataspaceunit.upm/realms/demo/protocol/openid-connect/certs>`
4. Kubernetes lo reinicia
5. Mientras se reinicia → 502 Bad Gateway

SOLUCIÓN: forzar un reinicio limpio

Ahora que Keycloak está estable, se fuerza un reinicio del conector para que arranque limpio:

```
minikube kubectl -- rollout restart deployment -n demo conn-citycouncil-demo
```

Y luego se espera a que esté listo:

```
minikube kubectl -- rollout status deployment -n demo conn-citycouncil-demo --timeout=180s
```

Una vez `successfully rolled out`, conviene esperar **30-60 segundos extra** para que el EDC complete su arranque interno, y verificar los logs:

```
minikube kubectl -- logs -n demo -l app.kubernetes.io/name=conn-citycouncil-demo --tail=30
```

Se debe buscar un mensaje como:


```
Runtime started
```

O

```
Started <connector-name>
```

Después, se vuelve a ejecutar el smoke test

```
curl -s -w "\nHTTP %{http_code}\n" -X POST "${PROVIDER_BASE}/management/v3/assets/request" \
-H "Authorization: Bearer ${PROVIDER_JWT}" \
-H "Content-Type: application/json" \
-d '{"@context":{"@vocab":"<https://w3id.org/edc/v0.0.1/ns/"}, "offset":0, "limit":1, "filterExpression":[]}'
```

Resultado esperado: **HTTP 200** con un JSON `[]` (array vacío) o con assets si hay alguno previo.

Si el reinicio no soluciona el problema

Deben revisarse los logs completos del último arranque para comprobar si hay un error nuevo (por ejemplo, conexión a PostgreSQL o Vault):

```
minikube kubectl -- logs -n demo -l app.kubernetes.io/name=conn-citycouncil-demo --tail=200
```

El primer paso debe ser el `rollout restart`; es muy probable que sea suficiente porque los crashes anteriores fueron causados por Keycloak inestable, y ahora todo el stack está sano.

Para el otro connector:

```
minikube kubectl -- rollout restart deployment -n demo conn-company-demo
minikube kubectl -- rollout status deployment -n demo conn-company-demo --timeout=180s
```

```
minikube kubectl -- logs -n demo -l app.kubernetes.io/name=conn-company-demo --tail=30
```

Plan de fases

Las tres fases del taller están enlazadas a sus páginas de teoría y metodología. Cada página tiene al final una sección **Comandos del taller** con los `curl` exactos que se ejecutan en directo.



AVISO: Antes de empezar cada fase desde una terminal nueva

Fase 3 suele arrancarse en una pestaña distinta a las de Fase 1/2.

Para evitar errores `mandatory value contractId is missing` y `405 Method Not Allowed`, antes del paso 3.1:

- Volver a ejecutar `source` sobre los `export_*.sh` (dataspace, suffix, provider, consumer).
- **Reexportar manualmente** los IDs que se generaron en F1/F2 y que no están en los `.sh`:

```
export AGREEMENT_ID="<uuid del paso 2.5>"
export ASSET_ID="asset-${SUFFIX}"
```

- Renovar `PROVIDER_JWT` y `CONSUMER_JWT` (ver callout "Renovar JWT como primer reflejo" del bloque "Smoke test inicial" más arriba).

Esta fue la causa #1 de fallos en el ensayo previo del taller.

Al final de cada sección hay un chunk para exportar variables a archivos y luego importarlos en la nueva terminal.

Fase 1 - Provider

Objetivo: dejar publicado en el catálogo del provider un asset dataset con su vocabulario, sus dos políticas y su contract definition.

Fase 1: Provider

Fase 2 - Consumer

Objetivo: descubrir desde el consumidor la oferta publicada, inspeccionar la política y completar la negociación contractual con `contractAgreementId` listo para transferir.

Fase 2: Consumer

Fase 3 - Transfer

Objetivo: lanzar la transferencia, seguir su estado, resolver el EDR y acceder de verdad al recurso publicado.

Fase 3: Transfer

Plantilla de registro de incidencias

Ficha mínima común a las tres fases. Conviene rellenar una por cada hallazgo.

Campo	Valor
Módulo	Auth / Vocabularios / Policies / Assets / Contract Definitions / Catálogo / Negociación / Transfer / EDR
Caso de prueba	Nombre corto
Endpoint	Ruta exacta usada
Esperado	Qué debía pasar
Observado	Qué pasó realmente
Clasificación	Bug / usabilidad / limitación / hueco documental
Severidad	Bloqueante / alta / media / baja
IDs clave	vocabId / policyId / assetId / cdId / negotiationId / agreementId / transferId
Evidencia	Respuesta HTTP, captura o log

Limpieza opcional al cierre

Borrado en orden inverso a la creación. La política del conector EDC tiene **dos limitaciones importantes** que conviene asumir antes de empezar:

- Los `contractAgreement` son **inmutables** (no se borran por API). Las negociaciones cerradas en talleres anteriores seguirán apareciendo en `Contracts` y `Transfer History`.

- Mientras exista un `contractAgreement` que apunte al asset, el `DELETE` del asset devuelve **HTTP 409** (`Asset ... cannot be deleted as it is referenced by at least one contract agreement`).

Paso previo: reexportar los IDs

La terminal del cierre suele ser distinta de las usadas en F1/F2/F3. Antes de los `DELETE`, se copian los IDs del resumen del paso `3.8 Resumen final del taller` (Fase 3: Transfer) y se reexportan. Ejemplo con valores ficticios, que deben sustituirse por valores reales:

```
export SUFFIX="1777382255"
#export VOCAB_ID="vocab-${SUFFIX}"
export VOCAB_ID="vocab-"
#export ACCESS_POLICY_ID="access-${SUFFIX}"
export ACCESS_POLICY_ID="access-"
#export CONTRACT_POLICY_ID="contract-${SUFFIX}"
export CONTRACT_POLICY_ID="contract-"
#export ASSET_ID="asset-${SUFFIX}"
export ASSET_ID="asset-"
export CD_ID="cd-${SUFFIX}"
export TRANSFER_ID="<uuid del paso 3.2>"

for v in SUFFIX VOCAB_ID ACCESS_POLICY_ID CONTRACT_POLICY_ID ASSET_ID CD_ID TRANSFER_ID; do
    echo "${v}=${(P)v}"
done
```

Y se renuevan el `PROVIDER_JWT` y el `CONSUMER_JWT` (el callout "Renovar JWT como primer reflejo" del bloque **Smoke test inicial** lo explica).



IMPORTANTE: Revisar las ids reales.

Limpieza por capas

Primero se terminan las transferencias activas; después se borran los recursos del lado provider de arriba a abajo:

```

if [ -n "${TRANSFER_ID}" ]; then
    curl -s -X POST "${CONSUMER_BASE}/management/v3/transferp
rocesses/${TRANSFER_ID}/terminate" \
        -H "Authorization: Bearer ${CONSUMER_JWT}" \
        -H "Content-Type: application/json" \
        -d '{"@context":{"@vocab":"https://w3id.org/edc/v0.0.1/
ns/"},"reason":"taller-cleanup"}' \
        -w "\nHTTP %{http_code}\n"
fi

curl -s -X DELETE "${PROVIDER_BASE}/management/v3/contractd
efinitions/${CD_ID}" \
    -H "Authorization: Bearer ${PROVIDER_JWT}" -w "\nHTTP %{h
ttp_code}\n"
curl -s -X DELETE "${PROVIDER_BASE}/management/v3/policydef
initions/${ACCESS_POLICY_ID}" \
    -H "Authorization: Bearer ${PROVIDER_JWT}" -w "\nHTTP %{h
ttp_code}\n"
curl -s -X DELETE "${PROVIDER_BASE}/management/v3/policydef
initions/${CONTRACT_POLICY_ID}" \
    -H "Authorization: Bearer ${PROVIDER_JWT}" -w "\nHTTP %{h
ttp_code}\n"
curl -s -X DELETE "${PROVIDER_BASE}/management/v3/assets/
${ASSET_ID}" \
    -H "Authorization: Bearer ${PROVIDER_JWT}" -w "\nHTTP %{h
ttp_code}\n"
curl -s -X DELETE "${PROVIDER_BASE}/management/vocabularie
s/${VOCAB_ID}" \
    -H "Authorization: Bearer ${PROVIDER_JWT}" -w "\nHTTP %{h
ttp_code}\n"

```



IMPORTANTE: Códigos HTTP esperables y cómo interpretarlos

- **204 No Content** → borrado correcto.
- **404 Not Found** → el recurso no existía o ya estaba borrado, seguir adelante.
- **405 Method Not Allowed** → el path llegó sin id (`/management/v3/.../`). Causa: variable vacía. Solución: reexportar el ID y reintentar.
- **409 Conflict en asset** → mensaje `Asset ... cannot be deleted as it is referenced by at least one contract agreement`. Es comportamiento normal de EDC: los `contractAgreement` son inmutables y mantienen al asset "vivo" aunque borres su contract definition y policies.

Limpieza total (solo si se necesita dejar el conector totalmente vacío): escalar a recrear el namespace `demo` o reiniciar la BD PostgreSQL del conector. Por API no es posible eliminar agreements. El procedimiento concreto vía PostgreSQL está en la sección **Reset completo del dataspace** justo debajo.

Reset completo del dataspace (limpieza nuclear vía PostgreSQL)

La limpieza por API que hay arriba es selectiva: borra `assets`, `policies`, `contractDefinitions` y `vocabularies`, pero **no puede borrar** los `contractAgreement`, `contractNegotiation` ni `transferProcess` ya generados. Por eso, después de un taller, `Transfer History` y `Contracts` siguen mostrando líneas residuales aunque se ejecute el cleanup completo.

Esta sección describe cómo dejar el dataspace `demo` **completamente virgen** (0 assets, 0 negociaciones, 0 transfers) vaciando directamente las dos bases de datos PostgreSQL del conector. Útil entre talleres si se necesita empezar de cero sin redespargar el namespace.



AVISO: Cuándo usar este reset y cuándo NO

- **SÍ:** antes de un taller nuevo si se necesita `Transfer History` y `Contracts` totalmente vacíos.
- **NO** si solo se necesita deshacer el último taller: se usa la **"Limpieza opcional al cierre"** de más arriba, ya que es menos invasiva.
- **NO** si se necesita reset total del namespace `demo`: eso requiere `kubectl delete namespace demo` + redeploy con Helm, fuera del alcance de esta sección.

Este procedimiento **no toca**:

- Vault (las claves criptográficas DSP del conector siguen vivas).
- MinIO (los buckets de datos de los assets S3 siguen intactos).
- Keycloak (los usuarios / clients siguen intactos).

Datos del entorno (despliegue local)

- **Pod PostgreSQL:** `common-srvs-postgresql-0` en namespace `common-srvs`.
- **Bases de datos del conector EDC:**
 - `conn_citycouncil_demo` (provider)
 - `conn_company_demo` (consumer)
- **Deployments backend EDC** (a escalar a 0): `conn-citycouncil-demo` y `conn-company-demo` en namespace `demo`. Los `*-inteface` son la UI del conector y no tocan la BD del EDC, no es necesario pararlos.

La password del usuario `postgres` se obtiene del secret en cada despliegue, sin hardcodearla:

```
export PG_PASSWORD=$(minikube kubectl -- get secret common-srvs-postgresql -n common-srvs -o jsonpath='{.data.postgres-password}' | base64 -d)
echo "PG_PASSWORD=${PG_PASSWORD:0:3}*** (len=${#PG_PASSWORD})"
```

Paso 1 - Escalar a 0 los backends EDC

```
minikube kubectl -- scale deployment conn-citycouncil-demo
-n demo --replicas=0
minikube kubectl -- scale deployment conn-company-demo -n d
emo --replicas=0

minikube kubectl -- get deployment -n demo -o custom-column
s='NAME:.metadata.name,REPLICAS:.spec.replicas,READY:.statu
s.readyReplicas'
```

Resultado esperado: las filas de `conn-citycouncil-demo` y `conn-company-demo` deben mostrar `REPLICAS=0` y `READY=<none>`. Si quedan en `1`, algo no se escaló; reintentar antes de continuar.

Paso 2 - Snapshot pre-truncate (estado actual)

Se lanza un conteo de las tablas principales para dejar registro del estado previo:

```
for db in conn_citycouncil_demo conn_company_demo; do
  echo "=== ${db} ==="
  minikube kubectl -- exec -n common-srvs common-srvs-postg
resql-0 -- \
    env PGPASSWORD="${PG_PASSWORD}" psql -U postgres -d ${d
b} -c "
SELECT 'edc_asset' AS tbl, COUNT(*) FROM edc_asset
UNION ALL SELECT 'edc_contract_definitions', COUNT(*) FROM
edc_contract_definitions
UNION ALL SELECT 'edc_policydefinitions', COUNT(*) FROM edc
_policydefinitions
UNION ALL SELECT 'edc_contract_negotiation', COUNT(*) FROM
edc_contract_negotiation
UNION ALL SELECT 'edc_contract_agreement', COUNT(*) FROM ed
c_contract_agreement
UNION ALL SELECT 'edc_transfer_process', COUNT(*) FROM edc_
transfer_process
UNION ALL SELECT 'edc_edr_entry', COUNT(*) FROM edc_edr_ent
ry
```



```

UNION ALL SELECT 'edc_vocabulary', COUNT(*) FROM edc_vocabulary;
"
done

```

Paso 3 - Truncate masivo de ambas DBs

Un solo `TRUNCATE` con todas las tablas EDC y `CASCADE` para que PostgreSQL resuelva el orden de las foreign keys automáticamente.

```

for db in conn_citycouncil_demo conn_company_demo; do
    echo "=== TRUNCATE ${db} ==="
    minikube kubectl -- exec -n common-srvs common-srvs-postgres-0 -- \
        env PGPASSWORD="${PG_PASSWORD}" psql -U postgres -d ${db} -c "
    TRUNCATE TABLE
        edc_accesstokendata,
        edc_asset,
        edc_catalog,
        edc_catalog_data_service,
        edc_contract_agreement,
        edc_contract_definitions,
        edc_contract_negotiation,
        edc_data_plane,
        edc_data_service,
        edc_dataset,
        edc_distribution,
        edc_edr_entry,
        edc_lease,
        edc_policydefinitions,
        edc_transfer_process,
        edc_vocabulary
    RESTART IDENTITY CASCADE;
    "
done

```



IMPORTANTE: Por qué funciona este TRUNCATE

- `RESTART IDENTITY` resetea las secuencias internas de PostgreSQL.
- `CASCADE` propaga el TRUNCATE a tablas con FK (`edc_dataset` , `edc_distribution` , `edc_catalog` , `edc_lease` , etc.) sin necesidad de ordenarlas manualmente.
- Hay que hacer **paso 1 primero** porque, si el conector estuviese arriba, mantendría locks abiertos sobre `edc_lease` y el TRUNCATE quedaría colgado.

Validado en este despliegue: 16 tablas EDC, 0 errores de FK al ejecutar la sentencia anterior.

Paso 4 - Verificación post-truncate

Se reejecuta el snapshot y se comprueba que **todas** las tablas estén a 0:

```
for db in conn_citycouncil_demo conn_company_demo; do
  echo "=== ${db} (post-truncate) ==="
  minikube kubectl -- exec -n common-srvs common-srvs-postg
resql-0 -- \
  env PGPASSWORD="${PG_PASSWORD}" psql -U postgres -d ${d
b} -c "
SELECT 'edc_asset' AS tbl, COUNT(*) FROM edc_asset
UNION ALL SELECT 'edc_contract_definitions', COUNT(*) FROM
edc_contract_definitions
UNION ALL SELECT 'edc_policydefinitions', COUNT(*) FROM edc
_policydefinitions
UNION ALL SELECT 'edc_contract_negotiation', COUNT(*) FROM
edc_contract_negotiation
UNION ALL SELECT 'edc_contract_agreement', COUNT(*) FROM ed
c_contract_agreement
UNION ALL SELECT 'edc_transfer_process', COUNT(*) FROM edc_
transfer_process
UNION ALL SELECT 'edc_edr_entry', COUNT(*) FROM edc_edr_ent
ry
UNION ALL SELECT 'edc_vocabulary', COUNT(*) FROM edc_vocabu
lary
```

```

UNION ALL SELECT 'edc_lease', COUNT(*) FROM edc_lease
UNION ALL SELECT 'edc_dataset', COUNT(*) FROM edc_dataset
UNION ALL SELECT 'edc_distribution', COUNT(*) FROM edc_distribution
UNION ALL SELECT 'edc_catalog', COUNT(*) FROM edc_catalog;
"
done

```

Si alguna fila no está a 0, parar y revisar (puede que el conector se haya re-escalado solo o que hubiera otra conexión abierta).

Paso 5 - Scale up de los conectores

```

minikube kubectl -- scale deployment conn-citycouncil-demo
-n demo --replicas=1
minikube kubectl -- scale deployment conn-company-demo -n d
emo --replicas=1

minikube kubectl -- rollout status deployment conn-citycoun
cil-demo -n demo --timeout=180s
minikube kubectl -- rollout status deployment conn-company-
demo -n demo --timeout=180s

```

Conviene esperar **30-60 segundos extra** después del rollout para que el EDC complete su arranque interno (los logs deben mostrar `Runtime started` o equivalente).

Paso 6 - Smoke test funcional

Se regeneran los JWTs (los anteriores ya no valen tras el reinicio del conector) y se comprueba que ambos lados responden con catálogo vacío:

```

export PROVIDER_JWT=$(curl -s -X POST "${KEYCLOAK_URL}/real
ms/${DS_NAME}/protocol/openid-connect/token" \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "grant_type=password" -d "client_id=${KC_CLIENT}" \
-d "username=${PROVIDER_USER}" -d "password=${PROVIDER_PA
SSWORD}" \
-d "scope=openid profile email" | jq -r .access_token)

```

```

export CONSUMER_JWT=$(curl -s -X POST "${KEYCLOAK_URL}/realms/${DS_NAME}/protocol/openid-connect/token" \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "grant_type=password" -d "client_id=${KC_CLIENT}" \
-d "username=${CONSUMER_USER}" -d "password=${CONSUMER_PASSWORD}" \
-d "scope=openid profile email" | jq -r .access_token)

echo "--- smoke provider ---"
curl -s -w "\nHTTP %{http_code}\n" -X POST "${PROVIDER_BASE}/management/v3/assets/request" \
-H "Authorization: Bearer ${PROVIDER_JWT}" -H "Content-Type: application/json" \
-d '{"@context":{"@vocab":"https://w3id.org/edc/v0.0.1/ns/"},"offset":0,"limit":1,"filterExpression":[]}'

echo "--- smoke consumer ---"
curl -s -w "\nHTTP %{http_code}\n" -X POST "${CONSUMER_BASE}/management/v3/assets/request" \
-H "Authorization: Bearer ${CONSUMER_JWT}" -H "Content-Type: application/json" \
-d '{"@context":{"@vocab":"https://w3id.org/edc/v0.0.1/ns/"},"offset":0,"limit":1,"filterExpression":[]}'

```

Resultado esperado (validado en directo):

```

--- smoke provider ---
[]
HTTP 200
--- smoke consumer ---
[]
HTTP 200

```

Dataspace virgen y conectores sanos, listos para empezar Fase 1 desde cero.